Intune App Deployment Troubleshooting & Failure Remediation (Win32/LOB/Store)

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary 4

The problem . . . 4

The solution . . . 4

What you get . . . 4

Business impact . . . 4

Bottom line . . . 5

Solution Overview 5

The problem this solves . . . 5

The app-delivery pipeline . . . 5

The ESP's role . . . 6

The diagnostic model . . . 6

What is in this solution . . . 6

Requirements 7

R1. Intune role that can read apps and device install status . . . 7

R2. Intune role that can modify apps and assignments (remediation only) . . . 7

R3. Microsoft Graph read access for the scripts . . . 8

R4. Device log access (Intune Management Extension logs) . . . 8

R5. A representative failing device (or device group) . . . 8

R6. PowerShell 5.1+ or PowerShell 7 with the Microsoft Graph SDK . . . 9

Global Requirements — Intune Solutions (category-level) 9

G1. Microsoft Entra ID tenant . . . 9

G2. Microsoft Intune subscription . . . 9

G3. Administrative roles . . . 9

G4. Device enrollment in Intune . . . 10

G5. Pilot security group . . . 10

G6. Supported Windows baseline . . . 10

G7. Network access to Microsoft cloud endpoints . . . 10

Diagnostic Playbook 10

Step 0 — Establish the facts (always do this first) . . . 10

Branch A — "Not applicable" (Requirement not met) . . . 11

Branch B — "Installed but shows Failed / not detected" (Detection wrong) . . . 11

Branch C — Blocked or looping install (Dependency / Supersedence) . . . 12

Branch D — Stuck downloading / "Waiting for install status" (Download failure) . . . 13

Branch E — Failed with a raw error code (Exit-code decode) . . . 13

Branch F — User stuck on the Enrollment Status Page (ESP block) . . . 14

After any fix — re-validate the whole pipeline . . . 14
Testing 15
Gate 1 — Requirement met (the app applies) . . . 15
Gate 2 — Installed (the installer succeeded) . . . 15
Gate 3 — Detected (Intune confirms it is present) . . . 16
Full-pipeline regression check . . . 16
Troubleshooting 16
Connecting to Microsoft Graph . . . 16
Reading the device logs . . . 17
Interpreting states . . . 17
Remediation and targeting scripts . . . 18
When the method stalls . . . 18
Best Practices 18
Detection rules — make them match reality . . . 18
Return codes and reboots . . . 19
Dependencies and supersedence — keep them shallow . . . 19
Phased rollout — pilot before fleet . . . 19
ESP discipline . . . 20
Operational hygiene . . . 20
Deliverables Summary 20
Documents . . . 20
PowerShell scripts . . . 21
How the pieces fit . . . 21
Appendix A — PowerShell Scripts 23

Executive Summary

The problem

Application deployment is where Intune most often fails visibly. A required Win32 app stalls at "Waiting for install status," reports "Not applicable," or fails with a raw error code — and the team cannot tell whether the cause is a requirement rule, a detection rule, a dependency, a supersedence chain, a download problem, the installer's own exit code, or the Enrollment Status Page. The usual response — retry, re-sync, re-create the app — burns hours, blocks users at first sign-on, and never identifies the real cause, so the failure returns.

The solution

This is a **root-cause method plus tooling** for Intune app deployment failures across Win32, LOB, and Store apps. It models every install as an ordered **pipeline** (assign → requirement → download → install → detection → report), provides a **symptom-driven diagnostic tree** that maps each observed symptom to the exact pipeline stage that owns it, and shows precisely where to read the **Intune Management Extension logs** for the definitive evidence at each stage. Five PowerShell scripts read the pipeline state directly from Microsoft Graph and make safe, gated fixes.

What you get

- A **pipeline model** and diagnostic method that turns a vague "install failed" into a named stage and a known cause.
- A **Diagnostic Playbook** covering the six failure branches — requirement not met, detection wrong, dependency/supersedence, download failure, exit-code decode, and ESP block — with exact checks and log locations for each.
- **Five PowerShell scripts** — read-only Discovery, Validation, and Report; and gated (**-WhatIf**) Remediation and pilot-targeting scripts.
- **Testing gates** (requirement met, installed, detected) that prove a fix, and **Best Practices** for detection rules and phased rollout that prevent recurrence.

Business impact

- **Faster resolution:** failures are traced to their real cause in minutes, not iterated on for hours.
- **Fewer blocked users:** ESP failures are diagnosed and cleared so enrollment completes.
- **Lower risk:** all fixes are piloted and validated before fleet rollout; remediation is gated.
- **Retained capability:** the method and scripts stay with your team and are reusable for every future app.

Bottom line

Every failed app install is traced to its real cause — requirement, detection, dependency, download, exit code, or ESP — and fixed, with a repeatable method and scripts you keep, at a fraction of the cost of ad-hoc troubleshooting or outside consulting.

Solution Overview

The problem this solves

A Win32 app you assigned in Intune sits at **"Waiting for install status,"** shows **"Not applicable,"** or fails with a raw error code — and nobody can say whether the cause is a **requirement rule**, a **detection rule**, a **dependency**, a **supersedence chain**, a **download failure**, the **installer's own exit code**, or the **Enrollment Status Page (ESP)**. People retry the install, re-sync the device, and re-create the app, hoping something sticks.

This solution replaces guessing with a **root-cause method**. It treats every app deployment as a **pipeline** with named stages, gives you a **symptom-driven diagnostic tree** that maps each observed symptom to the exact pipeline stage that owns it, and hands you **five PowerShell scripts** that read the state of that pipeline directly from Microsoft Graph — so you fix the real cause the first time.

It covers the three app types Intune delivers to Windows:

- **Win32 apps** (`.intunewin` packages delivered by the Intune Management Extension).
- **Line-of-business (LOB) apps** (a single `.msi` uploaded directly).
- **Microsoft Store apps** (new Store app type, delivered via winget).

The app-delivery pipeline

Every app install moves through the same ordered stages. A failure at any stage stops the ones after it, so **the first stage that fails is the root cause** — everything downstream is a symptom.

1. **Assign** — the app is targeted at a group (Required, Available, or Uninstall) and the device

is in that group's scope. If the device is not in scope, the app never arrives.

1. **Requirement check** — the Intune Management Extension evaluates the app's **requirement rules**

(OS version, architecture, disk space, registry/file/script checks). If a requirement is not met, the app reports **"Not applicable"** and never attempts to install. This is the single most misread state in Intune app deployment.

1. **Dependency / supersedence resolution** — if the app has **dependencies**, those must install

first; if the app **supersedes** another app, the older app is uninstalled or updated per the supersedence rule. A broken link here blocks or reorders the install.

1. **Download** — the Intune Management Extension downloads the encrypted content to

`C:\Program Files (x86)\Microsoft Intune Management Extension\Content` and decrypts it. Network, proxy, disk-space, or content-integrity problems fail this stage.

1. **Install** — the Management Extension runs the install command line. The installer returns an

exit code; Intune maps that code to Success, Failure, Soft reboot, Hard reboot, or Retry based on the app's **Return codes** table.

1. **Detection** — after a "success" exit code, Intune runs the app's **detection rules**

(MSI code, file, registry, or custom script) to confirm the app is actually present. If detection does not match, Intune reports the install as **failed** even though the installer succeeded — the classic "installs but shows failed" symptom.

1. **Report** — the device sends install status back to Intune, where it appears under the app's

Device install status and **Overview** reports, and (for the ESP) gates sign-in.

The ESP's role

The **Enrollment Status Page (ESP)** runs during Autopilot / first sign-in and can be configured to **block device use until selected apps install**. When an app in the ESP's **"Block device use until these required apps are installed"** list fails or times out, the user is stuck on the ESP with a generic error. The ESP does not introduce a new failure mode — it makes an existing pipeline failure (usually requirement, download, install, or detection) **fatal and visible at enrollment time**. Diagnosing an ESP block therefore means diagnosing the underlying app failure, plus checking the ESP's own **timeout**, **app selection**, and **assignment** settings.

The diagnostic model

The method has three moves, applied in order:

1. **Locate the failing stage.** Read the reported state (Not applicable, Waiting, Failed, error

code) and use the symptom tree in the Diagnostic Playbook to map it to one pipeline stage. Never fix a downstream stage before confirming the upstream ones passed.

1. **Read the evidence at that stage.** For every stage there is a definitive source of truth:

Graph reports the requirement/install/detection state per device, and the **Intune Management Extension logs** on the device (`C:\ProgramData\Microsoft\IntuneManagementExtension\Logs\IntuneManagementExtension.log` and `AppWorkload.log`) record the exact requirement result, download path, install command line, returned exit code, and detection result.

1. **Fix the cause, then re-validate the whole pipeline.** Correct the one stage, force a sync,

and confirm the app walks all the way to **Detected / Installed** — not just past the stage you touched.

What is in this solution

- **This overview** — the pipeline, the ESP's role, and the diagnostic model.
- **Requirements** (`02-requirements.md`) — the Intune roles, Graph read access, and device log

access you need, each with a verify step.

- **Diagnostic Playbook** (`03-implementation.md`) — the symptom tree with exact checks per branch,

including where to read the Management Extension logs.

- **Testing** (`05-testing.md`) — the pass/fail gates that prove an app is fixed.
- **Troubleshooting** (`06-troubleshooting.md`) — quick fixes for issues with the method and tools themselves.
- **Best practices** (`07-best-practices.md`) — detection-rule and phased-rollout discipline that prevents these failures.
- **Deliverables summary** (`08-deliverables-summary.md`) and the **executive summary**.
- **Five PowerShell scripts** — Discovery, Validation, Report, Remediation, and Set-ScopeTargeting.

Requirements

Before you run the Diagnostic Playbook or the scripts, confirm the items below. The **category-level global requirements** (Entra tenant, Intune subscription, enrolled devices, supported Windows baseline, network access) are listed in `global-requirements-intune-solutions.md` and are assumed here. This file covers the requirements specific to **troubleshooting app deployments**.

Each requirement is written as **Description / Why / Verify / Implement if missing.**

R1. Intune role that can read apps and device install status

Description: An Entra/Intune role that grants read access to **Apps**, their **assignments**, and per-device **install status**. **Intune Administrator** covers everything in this solution; **Help Desk Operator** or a custom role with **Managed apps** → **Read** and **Organization** → **Read** is enough for the read-only Discovery, Validation, and Report scripts.

Why: Every diagnostic step reads app configuration and install/detection state. Without app read rights the console shows nothing and the scripts return empty results.

Verify: Sign in to `https://intune.microsoft.com`, open **Apps** → **All apps**, and open any app's **Device install status**. If you can see the app list and per-device status, you have read access. For roles: Intune → **Tenant administration** → **Roles** → **All roles** and confirm your assignment.

Implement: Intune → **Tenant administration** → **Roles** → **All roles** → select **Help Desk Operator** (or **Intune Administrator** for remediation) → **Assignments** → **Assign** → add your account and a scope group. Assign remediation rights (R2) only to the operators who will run fixes.

R2. Intune role that can modify apps and assignments (remediation only)

Description: A role with **write** access to apps and assignments — **Intune Administrator** or a custom role with **Managed apps** → **Read/Update/Assign**.

Why: The Remediation and Set-ScopeTargeting scripts change assignments (for example, adding a pilot group) and, when you approve it, trigger re-attempts. The read-only scripts never need this.

Verify: In Intune, open an app → **Properties** → **Assignments** → **Edit**. If you can add a group and save, you have write access.

Implement: Assign **Intune Administrator**, or build a custom role (**Tenant administration** → **Roles** → **Create**) granting **Managed apps** Read, Update, and Assign, and assign it to the remediation operators only.

R3. Microsoft Graph read access for the scripts

Description: Delegated Microsoft Graph permissions for the PowerShell scripts: `DeviceManagementApps.Read.All` and `DeviceManagementManagedDevices.Read.All` for the read-only scripts, plus `DeviceManagementApps.ReadWrite.All` for Remediation and Set-ScopeTargeting.

Why: The scripts read app definitions, assignments, and per-device install/detection state through Graph. Remediation writes assignments, so it needs the ReadWrite scope.

Verify: Run any read-only script; at the first `Connect-MgGraph` prompt, sign in and **consent** to the listed scopes. If it returns app data, access is working. You can also confirm granted delegated permissions in Entra → **Enterprise applications** → **Microsoft Graph Command Line Tools** → **Permissions**.

Implement: Install the SDK — `Install-Module Microsoft.Graph -Scope CurrentUser`. Each script calls `Connect-MgGraph` with the scopes it needs; an Intune Administrator can consent on first use. If your tenant requires admin consent for the Graph Command Line Tools app, have a Global Administrator grant it once.

R4. Device log access (Intune Management Extension logs)

Description: The ability to read the Intune Management Extension logs on an affected Windows device, at `C:\ProgramData\Microsoft\IntuneManagementExtension\Logs\` — chiefly `IntuneManagementExtension.log` and `AppWorkload.log`.

Why: These logs are the **only** place that records the exact requirement result, download path, install command line, returned exit code, and detection result. Graph reports the *state*; the logs explain *why*.

Verify: On a device with the app assigned, open `C:\ProgramData\Microsoft\IntuneManagementExtension\Logs\IntuneManagementExtension.log`. If the file exists and shows recent entries, you have log access. (`CMTrace` or `Notepad` both open it.)

Implement: Get local admin or remote-file access to a sample failing device, or collect the logs through **Intune** → **Devices** → **the device** → **Collect diagnostics**, which packages the Management Extension logs for download. Keep the folder path 'your-log-path' handy for the playbook.

R5. A representative failing device (or device group)

Description: At least one enrolled device that reproduces the failure, ideally in a small **pilot** security group.

Why: Diagnosis and validation both need a device whose real state you can read in Graph and whose logs you can open. Fixing against a live fleet without a control device makes it impossible to prove the fix worked.

Verify: Intune → **Devices** → **All devices**; confirm the device is present, **compliant**, and recently synced (**last check-in** within a day).

Implement: Add a sample device to a pilot group (Entra → **Groups**), assign the app to that group, and use `INTAPPDX-Set-ScopeTargeting.ps1` to manage the pilot targeting safely.

R6. PowerShell 5.1+ or PowerShell 7 with the Microsoft Graph SDK

Description: A workstation running Windows PowerShell 5.1 or PowerShell 7 with the **Microsoft.Graph** module installed.

Why: All five scripts use the Microsoft Graph PowerShell SDK.

Verify: Run `$PSVersionTable.PSVersion` (expect 5.1 or 7.x) and `Get-Module Microsoft.Graph -ListAvailable` (expect a version to be listed).

Implement: `Install-Module Microsoft.Graph -Scope CurrentUser`. Run the scripts with `powershell -ExecutionPolicy Bypass -File .\<script-name>.ps1`.

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

Diagnostic Playbook

This is the core of the solution: a **symptom-driven tree**. Start from what you observe, follow the branch, run the exact checks, and you land on the one pipeline stage that owns the failure. Fix that stage, then re-run the validation gates in `05-testing.md`.

Golden rule: the app pipeline is ordered — **Assign** → **Requirement** → **Dependency/Supersedence** → **Download** → **Install** → **Detection** → **Report**. The **first** stage that fails is the root cause; everything after it is a symptom. Never fix a later stage before confirming the earlier ones passed.

Step 0 — Establish the facts (always do this first)

1. **Read the reported state in Intune.** Apps → **All apps** → the app → **Device install status**.

Note the exact status text for the affected device: *Not applicable*, *Waiting for install status*, *Failed (0x…)*, *Installed but not detected*, or *Installed*.

1. **Read the state from Graph.** Run `INTAPPDX-Validation.ps1` for the device to get the app's

`installState`, `installStateDetail`, and any error code in one place — this is faster and more precise than the console for triage across several apps.

1. **Open the device logs.** On the device, open

`C:\ProgramData\Microsoft\IntuneManagementExtension\Logs\IntuneManagementExtension.log` and `AppWorkload.log`. Search for the app's **name** or its **application id**. The log tells you, in order, the requirement result, the download, the install command line, the returned exit code, and the detection result. Match that against the pipeline to see where it stopped.

Now pick the branch that matches your symptom.

Branch A — "Not applicable" (Requirement not met)

Symptom: The device shows **Not applicable**, or the log says the app is *not applicable* and it never attempts to download or install.

Owner stage: Requirement check.

Checks:

1. **Confirm the device is even in scope.** Run `INTAPPDX-Discovery.ps1` and confirm the device's

user or the device is in a group the app is **assigned** to (Required or Available). If not in scope, the correct status *is* Not applicable — fix the assignment, not the requirement.

1. **Read each requirement rule.** In Intune, open the app → **Properties** → **Requirements**. Compare

every rule against the device's real state:

• **Operating system architecture** — 32-bit vs 64-bit. A 64-bit-only requirement makes the app

Not applicable on 32-bit OS.

• **Minimum operating system** — the device build must be equal or higher.

• **Disk space / physical memory / logical processors** — read the device's Hardware in Intune.

• **Registry / file / script requirement rules** — the custom check must return the value the

rule expects. In the log, find the requirement evaluation line and read the actual value the device returned.

1. **Find the exact rule that failed in the log.** `IntuneManagementExtension.log` /

`AppWorkload.log` logs the requirement evaluation, including which rule returned false and the value it saw. That line names the wrong rule directly.

Fix: Correct the requirement rule (loosen the OS/architecture rule, or fix the registry/file/ script check to match reality), or fix the assignment so the device is in scope. Bump nothing else.

Branch B — "Installed but shows Failed / not detected" (Detection wrong)

Symptom: The app is clearly installed on the device, but Intune reports **Failed** or keeps **re-installing** it every cycle. The install log shows a **success** exit code, then a detection line that does not match.

Owner stage: Detection.

Checks:

1. **Read the detection rule.** App → **Properties** → **Detection rules**. Detection can be **MSI**,

File, **Registry**, or a **custom detection script**.

1. **Verify the rule against the installed app on the device:**

• **MSI product code** — run `Get-WmiObject Win32_Product` (or check

`HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall`) and confirm the product code the rule expects is actually the one the installer registered. Repackaged or updated MSIs often ship a new product code.

• **File** — confirm the exact **path**, **file name**, **version** operator, and **32-bit vs

64-bit** program-files location. A file-version rule fails if the shipped version differs from the rule by even a build number, or if the rule points at `Program Files` on a 32-bit app that installed to `Program Files (x86)`.

• **Registry** — confirm the key, value name, data, and the **32-bit registry view** checkbox.

• **Custom script** — the script must **write a value to STDOUT and exit 0** to signal detected.

Run it manually on the device as SYSTEM (via `PsExec -s` or a scheduled task) and confirm it prints on success and prints nothing on failure.

1. **Confirm in the log.** The detection result is logged right after the install exit code. If the

exit code is success and detection is false, the detection rule — not the installer — is wrong.

Fix: Correct the detection rule so it matches what the installer actually leaves behind. Then force a sync and confirm the state moves to **Installed / Detected**.

Branch C — Blocked or looping install (Dependency / Supersedence)

Symptom: The app never starts, installs in the wrong order, or an older app keeps coming back / being removed unexpectedly. The log mentions a **dependency** app or a **superseded** app id.

Owner stage: Dependency / supersedence resolution.

Checks:

1. **List the relationships.** `INTAPPDX-Discovery.ps1` reports each app's **dependencies** and

supersedence links. Confirm they are what you intend.

1. **Dependency checks:**

• Every **dependency** app must itself be **assigned** (or set to **auto-install**) and must

succeed first. A dependency that is Not applicable or Failed on the device blocks the dependent app. Diagnose the dependency using Branch A/D/E/F as its own case.

• Watch for **circular dependencies** (A depends on B, B depends on A) — the log shows the chain

failing to resolve.

1. **Supersedence checks:**

• Confirm the supersedence mode: **Update** (replace in place) vs **Uninstall previous version**.

• A **too-deep supersedence chain** (more than one hop) is not supported and stalls resolution —

flatten it.

• Confirm the superseded app's **uninstall command line** actually works; if uninstall fails,

the new app cannot proceed.

Fix: Assign the missing dependency, break the circular link, flatten the chain to a single hop, or correct the uninstall command. Re-validate the whole pipeline for both apps.

Branch D — Stuck downloading / "Waiting for install status" (Download failure)

Symptom: The app sits at **Waiting for install status** for a long time; the log shows the Management Extension **downloading** content, retrying, or failing to decrypt/extract.

Owner stage: Download.

Checks:

1. **Read the download lines in the log.** In `IntuneManagementExtension.log`, find the content

download for the app id. Look for HTTP errors, timeouts, hash/decryption failures, or repeated retries.

1. **Disk space.** Confirm the system drive has room for the encrypted download **and** the

extracted payload under `C:\Program Files (x86)\Microsoft Intune Management Extension\Content`.

1. **Network / proxy.** The Management Extension must reach the Intune content delivery endpoints.

Confirm the device can reach the Microsoft cloud endpoints (see global requirement G7); a proxy that blocks the content CDN causes silent, repeated download failures.

1. **Content integrity.** A decryption/hash-mismatch error usually means the `.intunewin` needs to

be **re-uploaded** (repackage from source and replace the content).

Fix: Free disk space, open the required endpoints through the proxy, or re-upload the package. Then re-sync and watch the download complete in the log.

Branch E — Failed with a raw error code (Exit-code decode)

Symptom: The app shows **Failed (0x…)** or a decimal error; the log shows the install command line ran and the installer **returned a non-success exit code**.

Owner stage: Install (exit-code mapping).

Checks:

1. **Find the exit code in the log.** `AppWorkload.log` logs the exact **install command line** and

the **returned exit code** for the app. That code is the installer's, not Intune's.

1. **Decode the code:**

• **0** = success; **1707** = success; **3010** = success, soft reboot required; **1641** =

success, hard reboot initiated; **1618** = another install is in progress (retry); **1603** = fatal MSI error (the most common real failure).

- For **1603** and other MSI failures, re-run the install manually on the device with verbose

MSI logging (`msiexec /i package.msi /l*v install.log`) and read the failing action.

- For non-MSI installers, look up the vendor's documented exit codes.

1. **Check the app's Return codes table.** App → **Properties** → **Return codes**. If the installer

returns a code that Intune treats as **Failure** but the vendor documents it as success (or as "reboot required"), map it correctly in the table.

Fix: Resolve the installer's real error (from the verbose log), or correct the Return codes mapping so a benign code is not treated as failure. Re-run and confirm success + detection.

Branch F — User stuck on the Enrollment Status Page (ESP block)

Symptom: During Autopilot / first sign-in, the device is stuck on the **ESP** with an app error, or the ESP times out.

Owner stage: ESP (which is exposing an underlying pipeline failure).

Checks:

1. **Identify the blocking app.** Intune → **Devices** → **Enrollment** → **Enrollment Status Page** →

your ESP profile → the **"Block device use until these required apps are installed"** list. Whichever listed app is failing is the blocker.

1. **Diagnose that app** using Branches A–E — the ESP does not create a new failure; it makes an

existing requirement/download/install/detection failure fatal at enrollment. Read the same Management Extension logs on the device (they exist during ESP).

1. **Check the ESP settings themselves:**

- **Timeout** — too short a timeout fails a legitimately slow install.

- **App selection** — an app in the block list that is **Not applicable** to that hardware will

block forever; only list apps that apply to every enrolling device.

- **Assignment** — the ESP profile and every blocking app must be **assigned** to the enrolling

device's group.

1. **"Show error when installation takes longer than…"** — read the on-screen error and correlate it

to the log.

Fix: Resolve the underlying app failure, remove non-universal apps from the block list, extend the timeout if the install is legitimately long, and confirm every blocking app is assigned to the enrollment group. Re-image a test device to confirm the ESP completes.

After any fix — re-validate the whole pipeline

Do not stop at the stage you fixed. Force a sync (**Settings** → **Accounts** → **Access work or school** → **Info** → **Sync**, or `INTAPPDX-Remediation.ps1`), then re-run `INTAPPDX-Validation.ps1` and confirm the app reaches **Installed / Detected** — proving every downstream stage now passes too. Use the gates in `05-testing.md`.

Testing

Use these gates to prove an app deployment is actually fixed — not just that it moved past one stage. An app is only "fixed" when it passes **all three** gates on a representative device. Run `INTAPPDX-Validation.ps1` to read each state from Graph, and confirm against the device logs and the Intune console.

Gate 1 — Requirement met (the app applies)

What it proves: the device is in scope and every requirement rule passes, so the app will attempt to install.

Pass:

- Intune → app → **Device install status** shows the device as anything other than **Not applicable** (it should progress to installing/installed).
- `INTAPPDX-Validation.ps1` reports the device is targeted by an assignment and the requirement state is not "not applicable."
- The Management Extension log shows the requirement evaluation returning **applicable / true**.

Fail: Device shows **Not applicable** — return to Diagnostic Playbook **Branch A**.

Gate 2 — Installed (the installer succeeded)

What it proves: content downloaded and the installer returned a success exit code.

Pass:

- `AppWorkload.log` shows the install command line running and returning a **success** code (0, 1707, 3010, or 1641).
- Intune → **Device install status** advances past **Waiting for install status**.
- The application's files/registry entries are present on the device.

Fail:

- Stuck at **Waiting for install status** with download activity → **Branch D**.
- **Failed (0x…)** with a non-success exit code → **Branch E**.
- Blocked by a dependency/supersedence link → **Branch C**.

Gate 3 — Detected (Intune confirms it is present)

What it proves: the detection rule matches the installed app, so Intune reports success and stops re-installing.

Pass:

- Intune → **Device install status** shows **Installed**.
- `INTAPPDX-Validation.ps1` reports `installState = installed` and detection satisfied.
- The Management Extension log shows the detection rule returning **detected** immediately after the success exit code, and the app is **not** re-attempted on the next cycle.

Fail: Installer succeeded but status is **Failed** or the app re-installs every cycle → **Branch B** (detection wrong).

Full-pipeline regression check

After a fix, run this end-to-end on a pilot device:

1. Force a sync (`INTAPPDX-Remediation.ps1` or the device **Sync** button).
2. Run `INTAPPDX-Validation.ps1` for the device.
3. Confirm all three gates pass in one run.
4. Run `INTAPPDX-Report.ps1` for the app and confirm the pilot group's **coverage** improves and the **failure** count for that app drops.

Only when Gates 1–3 all pass on the pilot should you expand the assignment to the broader fleet.

Troubleshooting

This guide covers problems with the **method, scripts, and access** used in this solution. For diagnosing an actual app deployment failure, use the **Diagnostic Playbook** (`03-implementation.md`) — that is the symptom tree for the apps themselves. This file is about getting the tooling and the diagnostic process itself to work.

Connecting to Microsoft Graph

`Connect-MgGraph` **fails or the module is missing.**

- Install the SDK: `Install-Module Microsoft.Graph -Scope CurrentUser`.
- Confirm it is present: `Get-Module Microsoft.Graph -ListAvailable`.
- Run PowerShell as your admin account and re-run the script; consent to the requested scopes at the sign-in prompt.

"Insufficient privileges" or consent is required.

- The read-only scripts need `DeviceManagementApps.Read.All` and `DeviceManagementManagedDevices.Read.All`; Remediation/Set-ScopeTargeting also need `DeviceManagementApps.ReadWrite.All`.
- If your tenant blocks user consent, have a Global Administrator grant admin consent for the Microsoft Graph Command Line Tools app once (Entra → **Enterprise applications**).

Scripts return no data even though apps exist.

- Confirm your account has an Intune role that can read apps (R1) and that you signed in to the correct tenant. Re-run `Connect-MgGraph -TenantId your-tenant-id` if you belong to multiple tenants.

Reading the device logs

The Intune Management Extension logs are not where expected.

- Path is `C:\ProgramData\Microsoft\IntuneManagementExtension\Logs\` — `ProgramData` is hidden; type the full path into Explorer or use `CMTrace`/Notepad.
- Chief files: `IntuneManagementExtension.log` and `AppWorkload.log`. Older entries roll into numbered copies — check those for a failure that happened earlier.

No log entries for the app at all.

- The device may not be in scope (Not applicable / never assigned). Confirm with `INTAPPDX-Discovery.ps1` first.
- Force a sync (`INTAPPDX-Remediation.ps1` or device **Sync**) and check again after the next cycle.

Cannot access the device locally.

- Use Intune → **Devices** → **the device** → **Collect diagnostics** to package the Management Extension logs for download, then read them offline.

Interpreting states

Console and script disagree on status.

- The console can lag; Graph (via `INTAPPDX-Validation.ps1`) reflects the last reported state. Force a sync and re-read; treat the device log as the final authority.

Status keeps flipping between Installed and Failed.

- This is the detection-rule symptom — go to Diagnostic Playbook **Branch B**. It is not a tooling problem.

Everything shows "Not applicable."

• Confirm the device is in a group the app is assigned to (Discovery). If it is genuinely out of scope, Not applicable is correct — adjust the assignment, don't chase requirement rules.

Remediation and targeting scripts

Remediation made no change.

• By design: both `INTAPPDX-Remediation.ps1` and `INTAPPDX-Set-ScopeTargeting.ps1` default to **-WhatIf** and only print the planned change. Re-run with `-WhatIf:$false` after reviewing it to apply.

Remediation cannot modify the assignment.

• Read-only rights are enough for Discovery/Validation/Report but not for changes. Assign **Intune Administrator** (or Managed apps Update/Assign) per requirement R2 and re-consent to the ReadWrite scope.

A fix looks applied in Intune but the device hasn't changed.

• Devices apply on their sync cycle. Force a sync and allow a cycle, then re-run `INTAPPDX-Validation.ps1`. Do not re-create the app or re-run remediation repeatedly in a tight loop — read the log to confirm the app moved forward.

When the method stalls

• **You fixed a stage and it still fails.** You likely fixed a downstream stage while an upstream one is still failing. Return to Step 0 of the playbook, re-read the log in pipeline order, and fix the **first** failing stage.

• **The failure is intermittent across devices.** Compare a passing device's log to a failing one at the same stage; the difference (OS build, architecture, disk space, proxy, existing app version) is your cause.

• **Vendor-installer specifics.** Intune-side diagnosis stops at the exit code; decode installer-specific exit codes and MSI verbose logs against the vendor's own documentation.

Best Practices

These practices prevent the failures the Diagnostic Playbook exists to fix. Apply them when you package and assign apps, and most deployments never reach a symptom branch.

Detection rules — make them match reality

• **Detect what the installer actually leaves behind.** Before you author a detection rule, install

the app on a clean device and confirm the exact product code, file path + version, or registry value it creates. Author the rule from the real artifact, not from the vendor's documentation.

• **Prefer stable artifacts.** A registry value or a versioned key under `Uninstall` is more stable than a loose file whose path or name can change between builds.

• **Mind the 32-bit vs 64-bit split.** Point file rules at `Program Files` vs `Program Files (x86)` correctly, and set the **32-bit registry view** checkbox to match where the app writes.

• **Use version operators deliberately.** "Greater than or equal to" survives minor updates; an "equals" rule breaks the moment the app auto-updates and must be maintained.

• **Custom detection scripts:** write a value to STDOUT and **exit 0** to signal detected; print nothing on not-detected. Test the script as SYSTEM before shipping it.

• **Keep detection and requirement rules distinct.** Requirement = "should this install?"; detection = "did it install?". Mixing them causes the "Not applicable" and "installs but shows failed" confusion.

Return codes and reboots

• **Map benign codes correctly.** Add the installer's documented success/reboot codes to the app's **Return codes** table so a soft-reboot code (3010) is not mistaken for a failure.

• **Handle reboots explicitly.** For apps that require a reboot to finish, use the correct return-code mapping rather than forcing a hard reboot mid-deployment.

Dependencies and supersedence — keep them shallow

• **Keep supersedence to a single hop.** Chains deeper than one hop are unsupported and stall. Retire old versions cleanly rather than stacking supersedence.

• **Assign dependencies too.** A dependency must be assigned (or auto-install) and must succeed first; an unassigned or failing dependency silently blocks the dependent app.

• **Avoid circular links.** Never let two apps depend on each other.

Phased rollout — pilot before fleet

• **Always pilot first.** Use `INTAPPDX-Set-ScopeTargeting.ps1` to assign new or changed apps to a small **pilot** security group before the fleet. Validate all three gates on the pilot.

• **Expand in rings.** Pilot → early adopters → broad → all. Watch `INTAPPDX-Report.ps1` coverage and failure counts after each ring before widening.

• **Change one thing at a time.** When fixing a failing app, change the single root-cause setting,

re-validate, then move on — so you always know what fixed it.

- **Prefer Available over Required for optional apps.** Required apps that are Not applicable on some hardware generate noise; scope Required apps only to devices they truly apply to, especially in the ESP block list.

ESP discipline

- **Only block on universal apps.** Every app in the ESP "block device use" list must apply to **every** enrolling device; a Not-applicable app there blocks enrollment forever.
- **Set a realistic timeout.** Give slow-but-legitimate installs enough time; too short a timeout fails good deployments.
- **Test enrollment end to end.** Re-image a test device after any ESP or blocking-app change.

Operational hygiene

- **Keep the read-only scripts scheduled.** Run `INTAPPDX-Report.ps1` regularly to catch rising failure rates before users report them.
- **Read the log, don't guess.** The Management Extension log is authoritative for requirement, download, exit code, and detection — check it before changing settings.
- **Keep remediation gated.** Run `INTAPPDX-Remediation.ps1` and `INTAPPDX-Set-ScopeTargeting.ps1` in the default **-WhatIf** mode first, review the planned change, then re-run to apply.

Deliverables Summary

This solution delivers a complete, repeatable method plus tooling for diagnosing and fixing Intune app deployment failures (Win32, LOB, and Store apps).

Documents

Document	What it gives you
Solution Overview (`01-solution-overview.md`)	The app-delivery pipeline (assign → requirement → download → install → detection → report), the ESP's role, and the three-move diagnostic model.
Global Requirements (`global-requirements-intune-solutions.md`)	Category-level prerequisites (tenant, Intune, enrollment, Windows baseline, network).
Requirements (`02-requirements.md`)	Solution-specific prerequisites — Intune roles, Graph read access, device log access — each with Description / Why / Verify / Implement.

Document	What it gives you
Diagnostic Playbook (`03-implementation.md`)	The symptom tree: requirement not met, detection wrong, dependency/supersedence, download failure, exit-code decode, ESP block — with exact checks per branch and where to read the Management Extension logs.
Testing (`05-testing.md`)	The three pass/fail gates: requirement met, installed, detected — plus a full-pipeline regression check.
Troubleshooting (`06-troubleshooting.md`)	Fixes for issues with the method, scripts, and access.
Best Practices (`07-best-practices.md`)	Detection-rule discipline and phased rollout that prevent these failures.
Executive Summary (`executive-summary.md`)	The business case in one page.

PowerShell scripts

All scripts use the Microsoft Graph PowerShell SDK. Discovery, Validation, and Report are strictly **read-only**. Remediation and Set-ScopeTargeting default to **-WhatIf** (they show the planned change and make no modification unless you re-run to confirm).

Script	Purpose	Access
INTAPPDX-Discovery.ps1	Inventory apps, their assignments, dependencies, and supersedence links.	Read-only
INTAPPDX-Validation.ps1	Per-device install + detection state for an app, mapped to the three gates.	Read-only
INTAPPDX-Report.ps1	App deployment coverage and failure report across the fleet.	Read-only
INTAPPDX-Remediation.ps1	Safe re-attempt / cleanup (force sync, targeted retry).	-WhatIf default
INTAPPDX-Set-ScopeTargeting.ps1	Build/adjust pilot group targeting for phased rollout.	-WhatIf default

How the pieces fit

1. Confirm **Requirements**.
2. Run **Discovery** to map the app and its assignments/relationships.
3. Follow the **Diagnostic Playbook** branch that matches your symptom; read the logs.
4. Apply the fix; run **Remediation** to re-attempt safely.
5. Prove it with **Testing** gates (via **Validation**), then track fleet health with **Report**.

6. Expand with **Set-ScopeTargeting**, following **Best Practices**.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

INTAPPDX-Discovery.ps1

Purpose: Builds a read-only inventory of Win32, LOB, and Store apps in the tenant. For each app it lists the assignment groups (Required/Available/Uninstall), dependency apps, and supersedence links so you can see where a device is in scope and whether a relationship is blocking an install. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\INTAPPDX-Discovery.ps1

How to run: powershell -ExecutionPolicy Bypass -File .\INTAPPDX-Discovery.ps1

INTAPPDX-Remediation.ps1

Purpose: For an app and the devices that report failed, triggers a device sync so the Intune Management Extension re-attempts the pipeline (download, install, detect). This is the safe re-attempt/cleanup step after you have corrected the root-cause setting per the Diagnostic Playbook. It changes nothing on the app definition. Runs in preview mode by default and only shows the planned actions. HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\INTAPPDX-Remediation.ps1 -AppNameLike "your-app-name" Apply: powershell -ExecutionPolicy Bypass -File .\INTAPPDX-Remediation.ps1 -AppNameLike "your-app-name" -WhatIf:$false

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\INTAPPDX-Remediation.ps1 -AppNameLike "your-app-name" Apply: powershell -ExecutionPolicy Bypass -File .\INTAPPDX-Remediation.ps1 -AppNameLike "your-app-name" -WhatIf:$false

INTAPPDX-Report.ps1

Purpose: Summarizes deployment health for each app: how many devices report installed, failed, not-applicable, and pending. Gives a coverage percentage and a ranked failure list so you can spot the apps that need attention before users report them. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\INTAPPDX-Report.ps1

How to run: powershell -ExecutionPolicy Bypass -File .\INTAPPDX-Report.ps1

INTAPPDX-Set-ScopeTargeting.ps1

Purpose: Adds a Required (or Available) assignment for a named pilot security group to an app, so you can validate a fix on a small ring before expanding to the fleet. It never removes existing assignments; it only adds the pilot targeting you name. Runs in preview mode by default and only shows the planned change. HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\INTAPPDX-Set-ScopeTargeting.ps1 -AppNameLike "your-app-name" -PilotGroupId your-group-id Apply: powershell -ExecutionPolicy Bypass -File .\INTAPPDX-Set-ScopeTargeting.ps1 -AppNameLike "your-app-name" -PilotGroupId your-group-id -Intent required -WhatIf:$false

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\INTAPPDX-Set-ScopeTargeting.ps1 -AppNameLike "your-app-name" -PilotGroupId your-group-id Apply: powershell -ExecutionPolicy Bypass -File .\INTAPPDX-Set-ScopeTargeting.ps1 -AppNameLike "your-app-name" -PilotGroupId your-group-id -Intent required -WhatIf:$false

INTAPPDX-Validation.ps1

Purpose: Reads the reported install state and detection detail for one app across the devices it targets (or for a single device you name). It maps each device to the three gates from the Testing guide: requirement met, installed, detected — so you can see exactly which pipeline stage a device stalled at. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\INTAPPDX-Validation.ps1 -AppNameLike "your-app-name"

How to run: powershell -ExecutionPolicy Bypass -File .\INTAPPDX-Validation.ps1 -AppNameLike "your-app-name"